

QUANTUM ENGINEERING QISKIT SIMULATIONS

Ken Zou

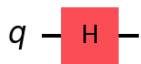
Class 142

The work consists of 11 exercises to solve using Qiskit from a very basic level to more advanced quantum protocols.

1 1-qubit circuits

1.1 Hadamard gate (Exercise 1 & 2)

The goal is to program a 1-qubit quantum circuit to create a superposition state so that
$$\begin{cases} |0\rangle \xrightarrow{H} \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle) = |+\rangle \\ |1\rangle \xrightarrow{H} \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle) = |-\rangle \end{cases}.$$



For that, we build a 1-qubit quantum circuit with a Hadamard gate H . We then show the transition from the initial quantum state to the final quantum state on the Bloch sphere. The H gate performs a 90° rotation around the Y -axis. The result is a 50/50 superposition where measurement yields $|0\rangle$ or $|1\rangle$ with equal probability.

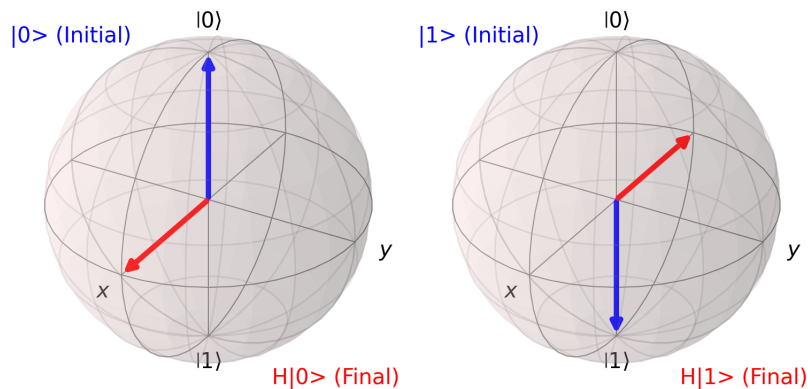


FIGURE 1 – Hadamard transition on the Bloch sphere for initial states $|0\rangle$ (left) and $|1\rangle$ (right).

1.2 Flip gate (Exercise 3)

The goal is to program a 1-qubit quantum circuit with a Flip gate (X gate) to an initial state and observe the result on the Bloch sphere :

$$\begin{cases} |0\rangle \xrightarrow{X} |1\rangle \\ |1\rangle \xrightarrow{X} |0\rangle \end{cases} .$$


The X gate performs a 180° rotation around the X -axis. This flips the state from $|0\rangle$ to $|1\rangle$ (or vice-versa), resulting in a 100% measurement probability for the flipped state.

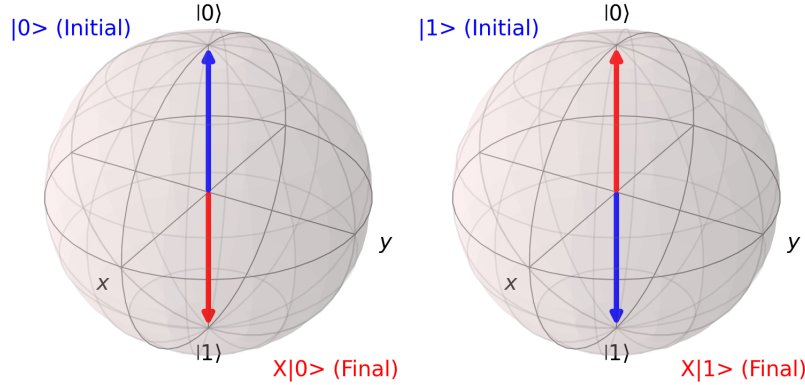


FIGURE 2 – Flip transition on the Bloch sphere for initial states $|0\rangle$ (left) and $|1\rangle$ (right).

1.3 Combining Flip and Hadamard gates (Exercise 4)

The goal is to program a 1-qubit quantum circuit so that we apply an X gate followed by an H gate :

$$\begin{cases} |0\rangle \xrightarrow{X} |1\rangle \xrightarrow{H} \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle) \\ |1\rangle \xrightarrow{X} |0\rangle \xrightarrow{H} \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle) \end{cases} \quad (1)$$



The result is the addition of the two previous operations : the X gate flips the initial state, and the H gate then transforms it into a superposition state. We note that quantum gates are non-commutative : reversing the order would yield a different final state ($|+\rangle$ instead of $|-\rangle$ for $|0\rangle$ for example). The result is still a 50/50 superposition where measurement yields $|0\rangle$ or $|1\rangle$ with equal probability, but the phases are different from the simple Hadamard gate.

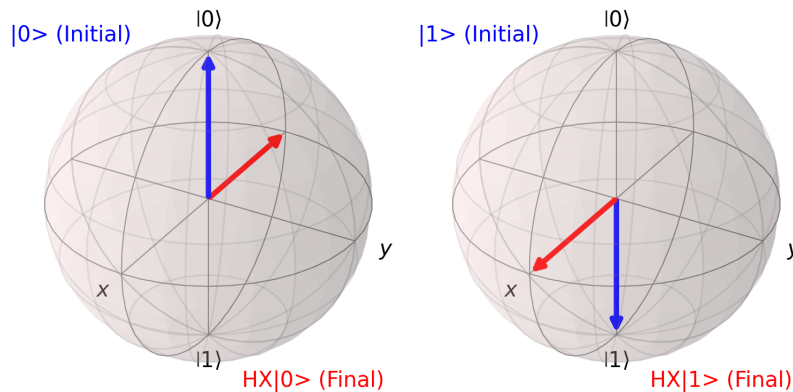


FIGURE 3 – HX transition on the Bloch sphere for initial states $|0\rangle$ (left) and $|1\rangle$ (right).

2 2-qubit circuits

2.1 Bell states β_{00} , β_{01} , β_{10} , β_{11} (Exercise 5 & 6)

The goal is to generate the four Bell states (β_{00} , β_{01} , β_{10} , β_{11}) using entanglement.

$$\begin{cases} |00\rangle \xrightarrow{H} \frac{1}{\sqrt{2}}(|00\rangle + |10\rangle) \xrightarrow{CNOT} \frac{1}{\sqrt{2}}(|00\rangle + |11\rangle) = \beta_{00} \\ |01\rangle \xrightarrow{H} \frac{1}{\sqrt{2}}(|01\rangle + |11\rangle) \xrightarrow{CNOT} \frac{1}{\sqrt{2}}(|01\rangle + |10\rangle) = \beta_{01} \\ |10\rangle \xrightarrow{H} \frac{1}{\sqrt{2}}(|00\rangle - |10\rangle) \xrightarrow{CNOT} \frac{1}{\sqrt{2}}(|00\rangle - |11\rangle) = \beta_{10} \\ |11\rangle \xrightarrow{H} \frac{1}{\sqrt{2}}(|01\rangle - |11\rangle) \xrightarrow{CNOT} \frac{1}{\sqrt{2}}(|01\rangle - |10\rangle) = \beta_{11} \end{cases} \quad (2)$$

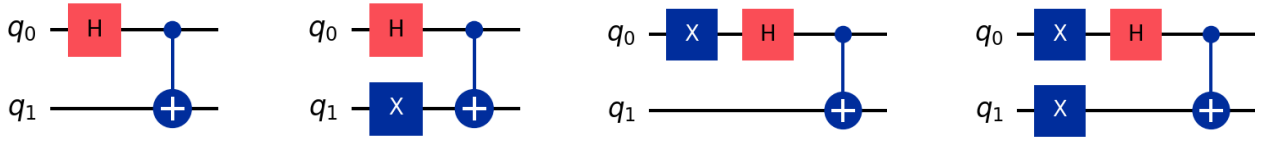


FIGURE 4 – Quantum circuit to generate, from left to right, β_{00} , β_{01} , β_{10} and β_{11} .

For that, we build a 2-qubit quantum circuit using a Hadamard gate followed by a CNOT gate to get a Bell state, which is an entangled state. We then try to show the initial quantum state and the final quantum state in the Bloch sphere, but it does not work. In fact, individual Bloch spheres cannot represent entangled qubits. In a Bell state, the qubits lack "pure" individual states because they are perfectly correlated. Mathematically, the reduced density matrix of an entangled qubit describes a "mixed state" which would appear as a point inside the sphere rather than a vector on the surface. This point does not provide the accurate piece of information.

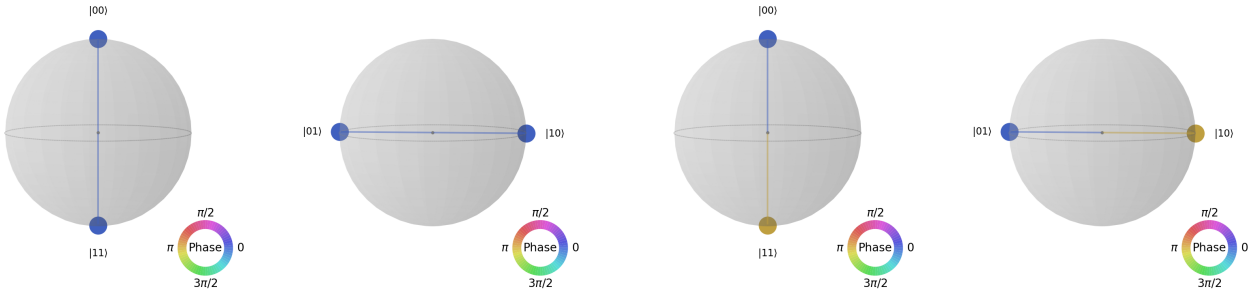


FIGURE 5 – Representation of the Bell states β_{00} , β_{01} , β_{10} and β_{11} with the associated phases on the QSphere

3 Combining quantum and classical register

3.1 3-qubit circuit (Exercise 7)

The goal is to program a 3-qubit quantum circuit so that the initial state is $|010\rangle$ and the final state $|001\rangle$. For that, following the Qiskit convention $|q_2q_1q_0\rangle$, we simply flip back q_0 and q_1 using X gates.

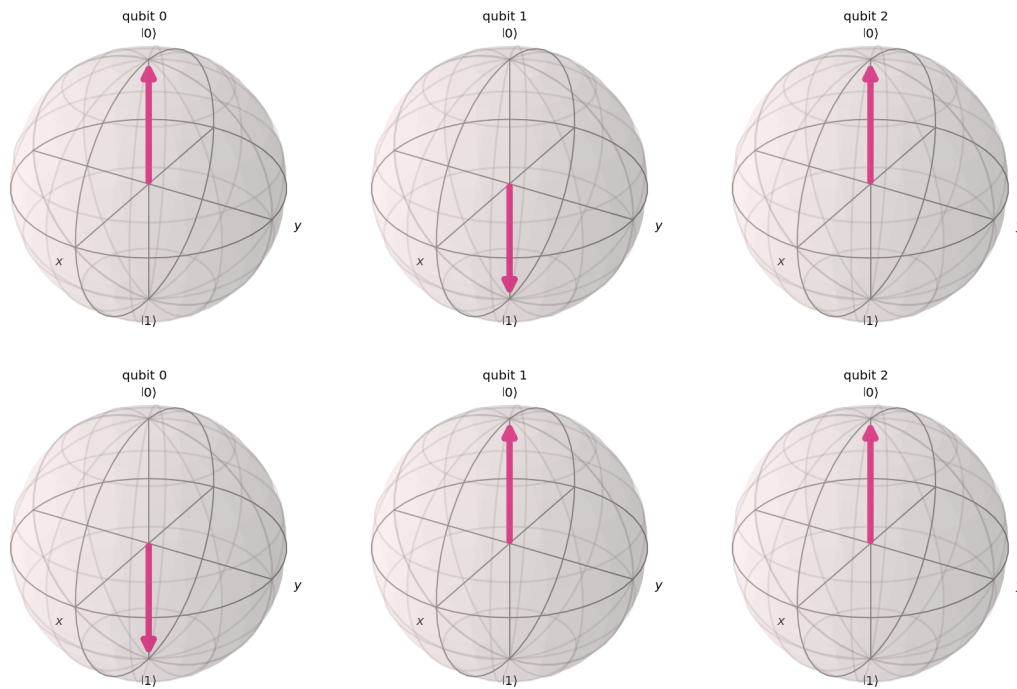


FIGURE 6 – Representation of the initial state $|010\rangle$ (up) and final state $|001\rangle$ (down) on the Bloch sphere

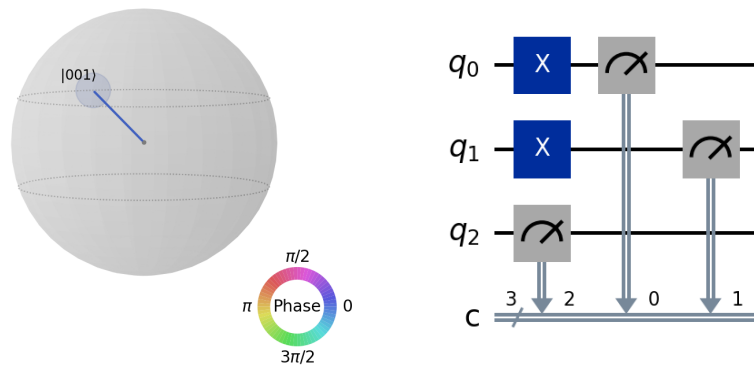


FIGURE 7 – Representation of the final state $|001\rangle$ on the QSphere and the associated quantum circuit

We then perform a simulation of the circuit. The results of the simulation are consistent with the first part of the exercise. The final state is $|001\rangle$ and the histogram shows a 100% measurement probability.

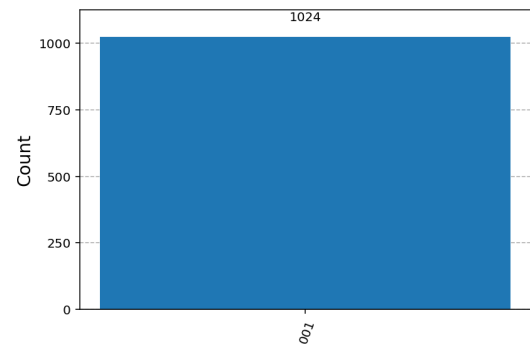


FIGURE 8 – Histogram of the results

3.2 1-qubit circuit with a random superposition state $|\psi\rangle$ (Exercise 8)

The goal is to generate a random superposition state $|\psi\rangle$ and use it as the input of a quantum circuit. We then measure the output at the end of the circuit.

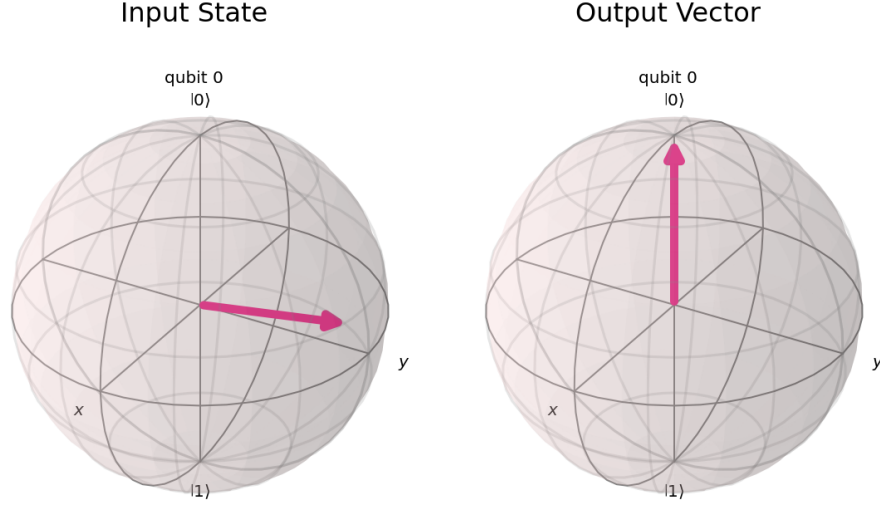
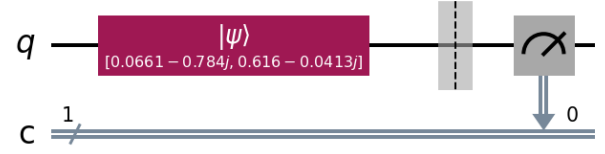


FIGURE 9 – Representation of the initial state (left) and the measurement (right) on the Bloch sphere

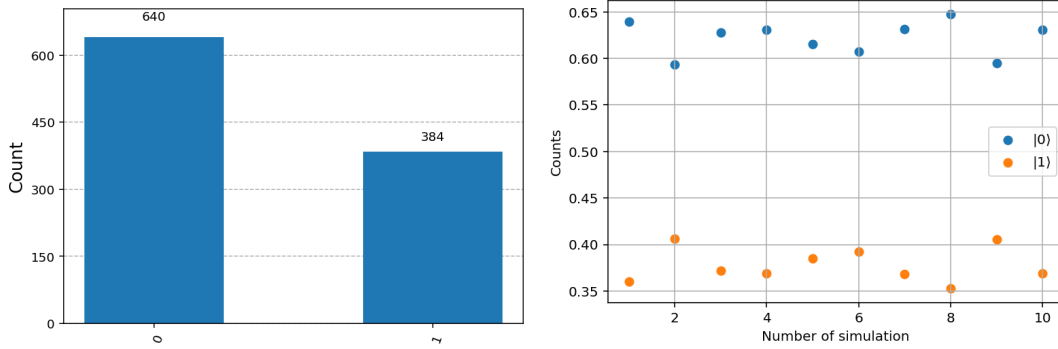


FIGURE 10 – Left : Histogram of the first simulation. Right : Count results when we repeat the simulation ten times (the counts are normalized to get the probability).

We either measure $|0\rangle$ or $|1\rangle$ with different probabilities. The histogram shows the number of times either $|0\rangle$ or $|1\rangle$ has been measured, which is closely linked to the measurement probability. We see that we measure $|0\rangle$ more frequently, which is consistent with the histogram and the expression of the initial state. Mathematically, with $|\psi\rangle = \alpha|0\rangle + \beta|1\rangle = (0.0661 - 0.784j)|0\rangle + (0.616 - 0.0413j)|1\rangle$, we should have the following probability

$$\begin{cases} |0\rangle \rightarrow |\alpha|^2 = (0.0661)^2 + (0.784)^2 = 0.619 \\ |1\rangle \rightarrow |\beta|^2 = (0.616)^2 + (0.0413)^2 = 0.381 \end{cases} \quad (3)$$

However, when repeating the simulation, the observed frequencies of $|0\rangle$ and $|1\rangle$ fluctuate around the theoretical probabilities. These fluctuations are known as shot noise or quantum statistical noise, which decreases as the number of simulation runs increases.

4 Quantum protocols

4.1 Quantum teleportation circuit (Exercise 9)

The goal is to program the quantum teleportation circuit. We suppose that Alice has a random qubit $|\psi\rangle$ that she wants to send to Bob. Also, Alice and Bob preshare a Bell state β_{00} and have classical communication.

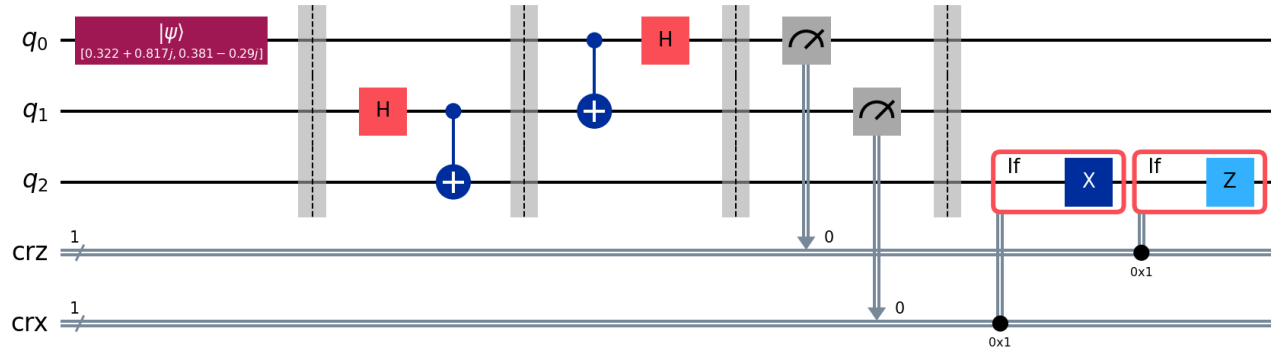


FIGURE 11 – Circuit architecture

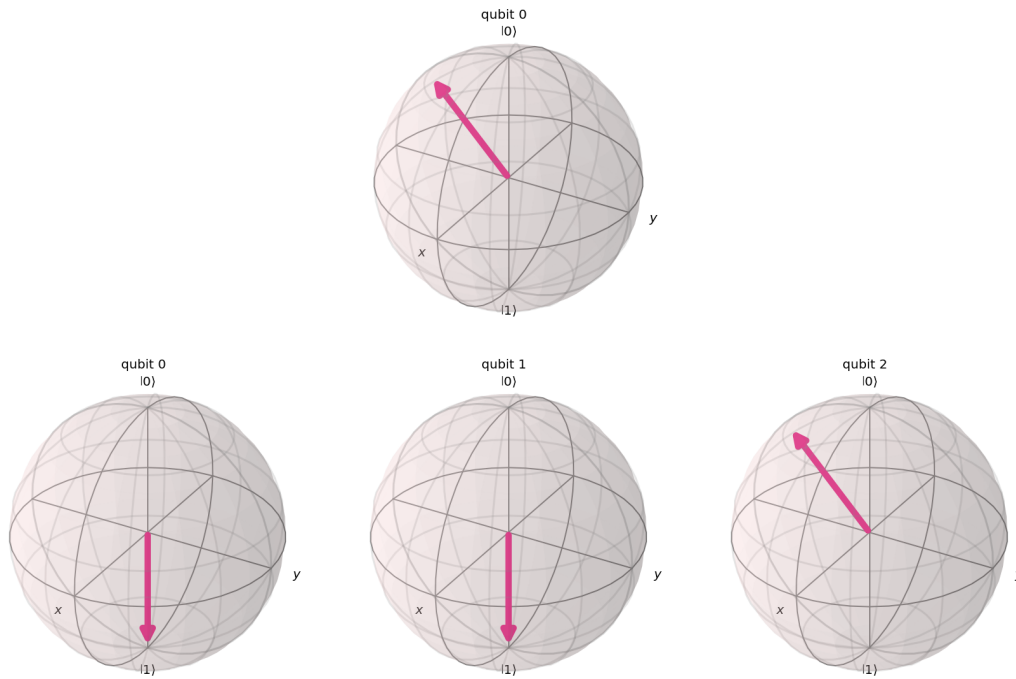


FIGURE 12 – Alice's state $|\psi\rangle$ (up) has been sent to Bob (q_2).

After Alice performs measurements on her qubits, Bob must apply specific corrections based on her results :

- if Alice measures $|00\rangle$, Bob does nothing since the third qubit q_2 is $|\psi\rangle$;
- if Alice measures $|01\rangle$, Bob performs CNOT gate to get $|\psi\rangle$;
- if Alice measures $|10\rangle$, Bob performs control-Z gate to get $|\psi\rangle$;
- if Alice measures $|11\rangle$, Bob performs both CNOT gate and control-Z gate to get $|\psi\rangle$.

We see that in all cases, the teleportation from q_0 (Alice) to q_2 (Bob) has been performed.

4.2 Grover's algorithm (Exercise 10)

Grover's algorithm finds the input state within an unstructured database of $N = 8$ elements. We use a 3-qubit system ($2^3 = 8$). The goal is to find a specific state out of the eight possibilities. Grover's algorithm increases the probability of measuring the correct answer. It consists of three main stages :

1. Initialization : We start by putting all qubits into a uniform superposition using Hadamard (H) gates. This means every state from $|000\rangle$ to $|111\rangle$ has the same probability amplitude.
2. The Oracle : This pins the correct answer by flipping its phase.
3. Inversion with respect to mean value : This increases the amplitude for the state solution. Other states are almost not affected. Repeating the two last operations several times increases the probability to obtain the state solution. For $N = 2^3 = 8$, the optimal number of iterations is approximately $\approx \frac{\pi}{4}\sqrt{2^3} \approx 2$.

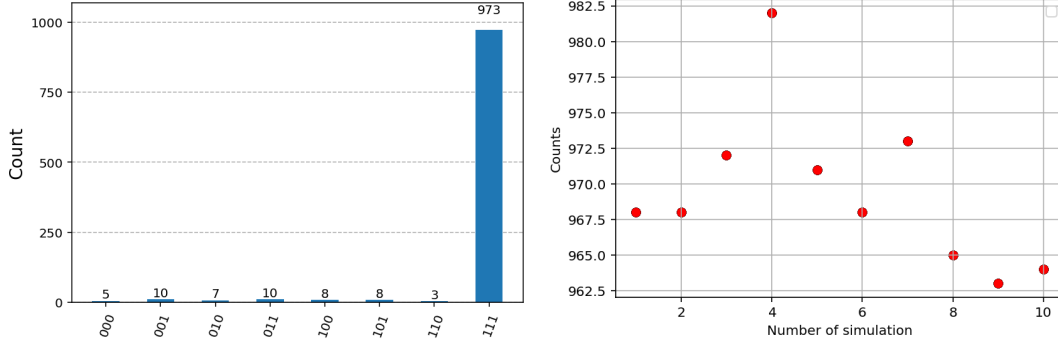


FIGURE 13 – Left : an example of histogram with the marked state $|111\rangle$. Right : count fluctuations for the marked state for multiple simulations.

We observe that for each simulation performed, there is a clear emphasis on the probability of the marked state, close to 95%. There is also shot noise.

4.3 BB84 quantum protocol (Exercise 11)

The BB84 protocol is a famous Quantum Key Distribution (QKD) scheme. It allows Alice and Bob to share a secret key that is mathematically guaranteed to be secure against eavesdropping.

To share that secret, photons with a given polarization are transmitted by Alice to Bob. There are two different orthogonal bases that can be used to encode 0 and 1 : (i) the rectilinear basis with $|0\rangle$ and $|1\rangle$ or (ii) the diagonal basis with $|+\rangle$ and $|-\rangle$.

The protocol works as follows :

1. Preparation : Alice generates a random string of bits and a random string of bases. She encodes each bit into a qubit using the chosen basis and sends them to Bob.
2. Measurement : Bob chooses his own random bases to measure the incoming qubits. Because of the Heisenberg Uncertainty Principle, if Bob chooses the wrong basis compared to Alice's (ie. Alice and Bob don't have the same basis), he cannot recover the original bit value with certainty.
3. Alice and Bob publicly announce which bases they used (but not the bit values). They keep only the bits where their bases matched.
4. Eavesdroppers introduce errors : Alice and Bob compare a small subset of their key to check for errors. If an eavesdropper tried to intercept the qubits, there are detectable errors. Note that an eavesdropper cannot measure a photon without disturbing them and therefore letting Bob know that the information is inaccurate, due to the no-cloning theorem.

To check if the protocol actually worked, we print Alice's key, Bob's key and we see if both keys match. In my program, the output is always **True**.

```

#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""
Created on Mon Mar 16 15:31:29 2026

@author: kenzou
"""

import numpy as np
import matplotlib.pyplot as plt
from qiskit import QuantumCircuit, QuantumRegister, ClassicalRegister, transpile
from qiskit_aer import Aer # Modern Aer import
from qiskit.visualization import plot_bloch_multivector, plot_state_qsphere, plot_state_qsphere
from qiskit.quantum_info import Statevector, SparsePauliOp, random_statevector
from IPython.display import display
from qiskit.visualization.bloch import Bloch
from qiskit.circuit.library import Initialize
from qiskit.circuit.library import ZGate

def Bloch_coordinates(state):
    x = state.expectation_value(SparsePauliOp('X')).real
    y = state.expectation_value(SparsePauliOp('Y')).real
    z = state.expectation_value(SparsePauliOp('Z')).real
    return [x, y, z]

#%% Exercise 1 & 2

def H_gate(qubit):
    initial_state = Statevector.from_label(str(qubit)) # Initial state
    QC = QuantumCircuit(1) # Quantum circuit with 1 qubit
    QC.h(0) # Hadamard gate
    final_state = initial_state.evolve(QC) # Final state
    print(f"The final state for input |{qubit}> is")

    display(final_state.draw('latex'))
    display(QC.draw('mpl', style={'title': "Hadamard circuit"})) # Drawing of t

    B = Bloch() # Bloch sphere
    B.add_vectors(Bloch_coordinates(initial_state))
    B.add_vectors(Bloch_coordinates(final_state))
    B.vector_color = ['blue', 'red']
    B.add_annotation([0.4, -0.7, 1.1], "|"+str(qubit)+"> (Initial)", color=B.vector_color[0])
    B.add_annotation([0.4, 1.1, -0.7], "H|"+str(qubit)+"> (Final)", color=B.vector_color[1])
    B.show()

    plt.show()

    return initial_state, final_state

H_gate(0)
H_gate(1)

#%% Exercise 3

def X_gate(qubit):

```



```

initial_state = Statevector.from_label(str(qubit)) # Initial state
QC = QuantumCircuit(1) # Quantum circuit with 1 qubit
QC.x(0) # The Flip gate performs the  $\pi$  rotation
final_state = initial_state.evolve(QC) # Final state

print(f"The final state for input |{qubit}> is")
display(final_state.draw('latex'))
display(QC.draw('mpl', style={'title': "Flip circuit"})) # Drawing of the c

B = Bloch()
B.add_vectors(Bloch_coordinates(initial_state))
B.add_vectors(Bloch_coordinates(final_state))
B.vector_color = ['blue', 'red']
B.add_annotation([0.4, -0.7, 1.1], "|" + str(qubit) + "> (Initial)", color=B.ve
B.add_annotation([0.4, 1.1, -0.7], "X|" + str(qubit) + "> (Final)", color=B.vec
B.show()
plt.show()

return initial_state, final_state

X_gate(0)
X_gate(1)

### Exercise 4

def H_X_gate(qubit):

    initial_state = Statevector.from_label(str(qubit)) # Initial state
    QC = QuantumCircuit(1)
    QC.x(0) # Flip
    QC.h(0) # Hadamard
    final_state = initial_state.evolve(QC) # Final state

    print(f"The final state for input |{qubit}> is")
    display(final_state.draw('latex'))
    display(QC.draw('mpl', style={'title': "1-qubit quantum circuit"})) # Drawi

    B = Bloch()
    B.add_vectors(Bloch_coordinates(initial_state))
    B.add_vectors(Bloch_coordinates(final_state))
    B.vector_color = ['blue', 'red']
    B.add_annotation([0.4, -0.7, 1.1], "|" + str(qubit) + "> (Initial)", color=B.ve
    B.add_annotation([0.5, 1, -0.7], "HX|" + str(qubit) + "> (Final)", color=B.vect
    B.show()
    plt.show()

    return initial_state, final_state

H_X_gate(0)
H_X_gate(1)

### Exercise 5 & 6

def Bell_state(bits):
    QC = QuantumCircuit(2) # 2-qubit circuit

```

```

if bits[0] == '1':
    QC.x(0)
if bits[1] == '1':
    QC.x(1)
# Standard Bell sequence
QC.h(0) # Hadamard
QC.cx(0, 1) # CNOT

print(f"The Bell state for input |{bits}> is")
display(QC.draw('mpl', style={'title': f"Circuit for |{bits}>"}))

state = Statevector.from_instruction(QC)
display(state.draw('latex'))

display(plot_state_qsphere(state)) # Show QSphere
plt.show()

# Test all four
for bits in ['00', '01', '10', '11']:
    Bell_state(bits)

### Exercise 7

# Qiskit order: |q2 q1 q0>
initial_state = Statevector.from_label('010') # Initial state |010>
QC = QuantumCircuit(3,3) # 3 qubits and 3 classical bits
QC.initialize(initial_state, [0, 1, 2])

display(plot_bloch_multivector(initial_state))

# Evolution to final state |001>
QC.x(1) # Flip q1 back to 0
QC.x(0) # Flip q0 to 1
final_state = initial_state.evolve(QC)

print("The final state is")
display(final_state.draw('latex'))

display(plot_bloch_multivector(final_state))
display(plot_state_qsphere(final_state))

QC.measure([0, 1, 2], [0, 1, 2])
display(QC.draw('mpl'))

sim = Aer.get_backend('aer_simulator')
QC.save_statevector()
out_vector = sim.run(QC).result().get_statevector()
display(plot_bloch_multivector(out_vector))
qobj = transpile(QC, sim)
result = sim.run(qobj).result()
counts = result.get_counts()
display(plot_histogram(counts))
plt.show()

### Exercise 8

```

```

psi = random_statevector(2) # random superposition state |psi>
print("The random statevector is")
display(psi.draw('latex'))
display(plot_bloch_multivector(psi, title="Input State"))

### Repeating the simulation (Exercise 8)

N = 10 # Number of simulations
L0 = []
L1 = []

for i in range(N) :

    # Initialization with the random state
    QC = QuantumCircuit(1, 1) # 1 qubit and 1 classical bit
    init_gate = Initialize(psi)
    QC.append(init_gate, [0])
    QC.barrier()

    QC.measure(0, 0)
    #display(QC.draw('mpl'))

    sim = Aer.get_backend('aer_simulator')
    QC.save_statevector()
    out_vector = sim.run(QC).result().get_statevector()
    #display(plot_bloch_multivector(out_vector, title="Output Vector"))
    qobj = transpile(QC, sim)
    result = sim.run(qobj).result()
    counts = result.get_counts()
    L1.append(list(counts.values())[0]/1024)
    L0.append(list(counts.values())[1]/1024)
    #display(plot_histogram(counts))
    plt.show()

plt.scatter(np.arange(1,N+1),L0,label = r"$\vert 0 \rangle$")
plt.scatter(np.arange(1,N+1),L1,label = r"$\vert 1 \rangle$")
plt.xlabel("Number of simulation")
plt.ylabel("Counts")
plt.grid(True)
plt.legend()
plt.show()

### Exercise 9

qr = QuantumRegister(3, name="q")
crz = ClassicalRegister(1, name="crz")
crx = ClassicalRegister(1, name="crx")
qc = QuantumCircuit(qr, crz, crx)

# Random state
psi = random_statevector(2)
qc.append(Initialize(psi), [0]) # This puts the random state on q0
qc.barrier()

```

```

# Entanglement
qc.h(1)
qc.cx(1, 2)
qc.barrier()

# Alice's operations
qc.cx(0, 1)
qc.h(0)
qc.barrier()

# Alice measures
qc.measure(0, crz) # Measure q0 into crz
qc.measure(1, crx) # Measure q1 into crx
qc.barrier()

# Bob's operations based on Alice's
# If crx is 1, Bob applies an X gate to q2
with qc.if_test((crx, 1)):
    qc.x(2)

# If crz is 1, Bob applies a Z gate to q2
with qc.if_test((crz, 1)):
    qc.z(2)

# Verification
sim = Aer.get_backend('statevector_simulator')
out_vector = sim.run(qc).result().get_statevector()

print("Original Random State (psi) on q0:")
display(plot_bloch_multivector(psi))

print("Final State on q2 after teleportation:")
display(plot_bloch_multivector(out_vector))

display(qc.draw('mpl'))
plt.show()

#%% Exercise 10

N = 10 # number of simulations

L = [[] for i in range(8)]

for k in range(N):

    # N = 8 => 3 qubits
    QC = QuantumCircuit(3)
    QC.h([0, 1, 2])
    QC.barrier()

    # For N=8, we run the Oracle + Inversion loop 2 times
    for i in range(2):
        # Oracle (marking |111>)
        QC.append(ZGate().control(2), [0, 1, 2])

```

```

    QC.barrier()

    # Inversion with respect to mean value
    QC.h([0, 1, 2])
    QC.x([0, 1, 2])
    QC.append(ZGate().control(2), [0, 1, 2])

    QC.x([0, 1, 2])
    QC.h([0, 1, 2])
    QC.barrier()

    QC.measure_all()

    # Simulation
    sim = Aer.get_backend('aer_simulator')
    result = sim.run(transpile(QC, sim)).result()
    counts = result.get_counts()

    l = list(counts.values())
    for i in range(8):
        L[i].append(l[i])

    #display(plot_histogram(counts))

plt.scatter(np.arange(1,N+1),L[-1],color = "red")
plt.xlabel("Number of simulation")
plt.ylabel("Counts")
plt.grid(True)
plt.legend()
plt.show()

#%% Exercise 11

def BB84(N_bits):
    # Alice generates random bits and bases
    alice_bits = np.random.randint(2, size=N_bits)
    alice_bases = np.random.randint(2, size=N_bits)

    # Alice prepares qubits
    QC = QuantumCircuit(N_bits, N_bits)
    for i in range(N_bits):
        if alice_bits[i] == 1: QC.x(i) # rectilinear
        if alice_bases[i] == 1: QC.h(i) # diagonal

    # Bob chooses random bases and measures
    bob_bases = np.random.randint(2, size=N_bits)
    for i in range(N_bits):
        if bob_bases[i] == 1: QC.h(i) # diagonal
    QC.measure(range(N_bits), range(N_bits))

    # Simulation
    sim = Aer.get_backend('aer_simulator')
    result = sim.run(transpile(QC, sim), shots=1).result()
    bob_results = [int(b) for b in list(result.get_counts().keys())[0][::-1]]

```

```

# Exchange of bases
final_key_alice = []
final_key_bob = []
for i in range(N_bits):
    if alice_bases[i] == bob_bases[i]:
        final_key_alice.append(alice_bits[i])
        final_key_bob.append(bob_results[i])

    return final_key_alice, final_key_bob

# Execute
N_bits = 20
alice_key, bob_key = BB84(N_bits)

print(f"Alice's key: {alice_key}")
print(f"Bob's key: {bob_key}")
print(f"Keys match: {alice_key == bob_key}")

```